

Spring Boot

Spring Web RESTful



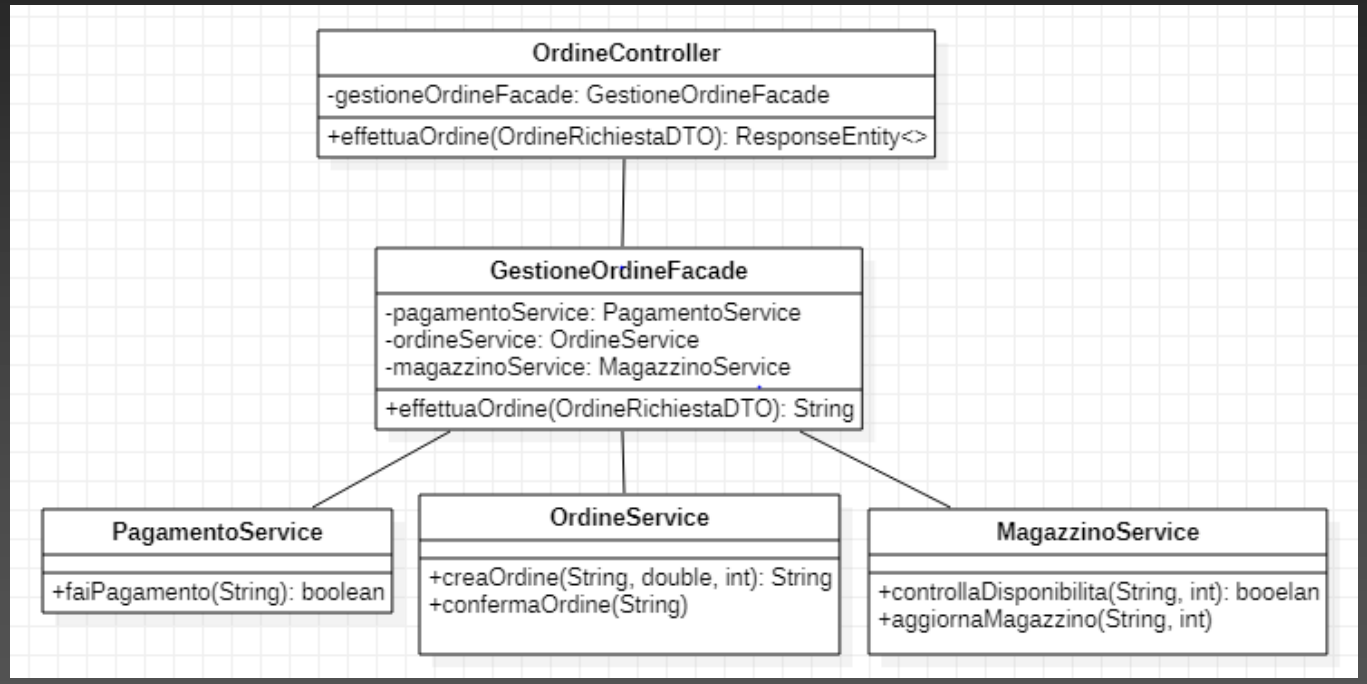
Facade Pattern



- Facade in inglese significa «facciata».
- E’ un design pattern strutturale.
- Usiamo un facade quando dobbiamo implementare un servizio complesso che internamente fa uso di altri servizi.
- Vantaggi:
  - Nasconde al componente che fa uso della facade la complessità che essa contiene.
  - Disaccoppia il componente che fa uso della facade dal servizio complesso. Le modifiche lasciano intatto il componente client.
  - Per usare un servizio interno della facade non dobbiamo per forza passare su di essa.



Facade Pattern



Facade Pattern



```java
@Service
@Transactional
public class PagamentoServiceImpl implements PagamentoService{

    @Override
    public boolean faiPagamento(String datiCarta) {
        // Business logic per effettuare il pagamento
        return true;
    }
}
```

```java
@Service
@Transactional
public class OrdineServiceImpl implements OrdineService{

    @Override
    public String creaOrdine(String prodotto, double prezzo, int quantita) {
        // Business logic per la creazione dell'ordine
        return "numero_ordine_xxx123";
    }

    @Override
    public void confermaOrdine(String numeroOrdine) {
        // Business logic per la conferma dell'ordine
    }
}
```

```java
@Service
@Transactional
public class MagazzinoServiceImpl implements MagazzinoService{

    @Override
    public boolean controllaDisponibilita(String prodotto, int quantita) {
        // Business logic per il controllo della disponibilità in magazzino
        return true;
    }

    @Override
    public void aggiornaMagazzino(String prodotto, int quantita) {
        // Business logic per l'aggiornamento della quantita di prodotto in magazzino

    }
}
```



Facade Pattern



```java
@Service
@Transactional
public class GestioneOrdineFacadeImpl implements GestioneOrdineFacade{

    @Autowired
    public MagazzinoService magazzinoService;
    @Autowired
    public PagamentoService pagamentoService;
    @Autowired
    public OrdineService ordineService;

    @Override
    public String effettuaOrdine(OrdineRichiestaDTO ordineDto) {
        boolean isDisponibile = magazzinoService
                .controllaDisponibilita(ordineDto.getProdotto(), ordineDto.getQuantita());
        String numeroOrdine = null;
        if(isDisponibile) {
            numeroOrdine = ordineService
                    .creaOrdine(ordineDto.getProdotto(), ordineDto.getPrezzo(), ordineDto.getQuantita());
        }else {
            throw new RuntimeException("Non disponibile nel magazzino");
        }
        boolean pagamentoAvvenuto = pagamentoService.faiPagamento(ordineDto.getDatiCarta());
        if(pagamentoAvvenuto) {
            ordineService.confermaOrdine(numeroOrdine);
            magazzinoService.aggiornaMagazzino(ordineDto.getProdotto(), ordineDto.getQuantita());
        }else {
            throw new RuntimeException("Errore durante il pagamento");
        }
        return numeroOrdine;
    }
}
```



Facade Pattern



```java
@RestController
@RequestMapping("/api/v1")
public class OrdineController {
        @Autowired
        public GestioneOrdineFacade service;

        @PostMapping("/ordine/crea")
        public ResponseEntity<Map<String, Object>> effettuaOrdine(@RequestBody OrdineRichiestaDTO ordineDto){
            String numeroOrdine = service.effettuaOrdine(ordineDto);
            Map<String, Object> responseBody = new HashMap<>();
            responseBody.put("timestamp", new Date());
            responseBody.put("status", HttpStatus.CREATED.value());
            responseBody.put("numero_ordine", numeroOrdine);
            return new ResponseEntity<>(responseBody, HttpStatus.CREATED);
        }

}
```



Facade Pattern



HTTP request

body

```
{
    "prodotto": "prodotto_xxx",
    "quantita": "2",
    "prezzo": "20.5",
    "datiCarta": "1234567891015347"
}
```

HTTP response body

```
{
    "numero_ordine": "numero_ordine_xxx123",
    "timestamp": "2022-10-05T08:48:08.165+00:00",
    "status": 201
}
```

spring boot